

[Context Course documentation](#)**Quiz 2: Hooks in Practice** ▾

Quiz 2: Hooks in Practice

[Copy page](#)

Test your ability to wire hooks into real workflows.

Question 1: How should the dashboard server be structured?

- ☐ The Gradio app and the FastAPI receiver must run as two separate processes on two separate ports
- ☒ Mounting Gradio on a FastAPI app with `gr.mount_gradio_app`` lets one server serve both the `/event`` endpoint and the Gradio UI

Correct! Correct! The FastAPI route `/event`` is defined before Gradio is mounted at `/``, so POSTs hit the receiver and everything else lands on the UI.

- ☐ You can only receive hook events from Claude Code — Codex, OpenCode, and Pi cannot post JSON
- ☐ Hook events should be written to a file rather than served over HTTP

[Submit](#)**You got all the answers!**

Question 2: How does the Gradio dashboard refresh?

- ☐ The dashboard polls with `gr.Timer(1.0)`` because browsers can't receive server-pushed events from Python
- ☒ Polling with `gr.Timer`` re-runs the refresh function every N seconds, which re-reads the shared `events`` buffer and re-renders the table and chart

Correct! Correct! Timer is the idiomatic Gradio pri

[Ask HuggingChat](#)

- ☐ The `gr.Timer` must be nested inside `gr.Row` or it won't fire
- ☐ The timer is mandatory — Gradio dashboards can't update any other way

Submit

You got all the answers!

Question 3: How do hook-based guardrails block dangerous actions?

- ☐ To block a dangerous Bash command in Claude Code, the hook returns HTTP 500
- ☒ A command hook that exits 2 and prints a reason on stderr is the documented way to block a tool call in Claude Code and Codex

Correct! Correct! Both platforms use the same convention: exit 2 + stderr means block, and the stderr message surfaces to the agent.

- ☐ In OpenCode, you block a tool by editing the user's prompt
- ☐ Guardrails should live in the model's system prompt rather than in hooks

Submit

You got all the answers!

Question 4: What makes shell-command hooks safe?

- ☐ Piping `curl --max-time 2` inside a Codex command hook is pointless; the agent never waits for hooks
- ☒ Shell command hooks can become a shell-injection risk if you interpolate JSON payloads into the command string — piping via `jq -c` and `curl --data-binary @-` is safer

Correct! Correct! Shape the payload with `jq` and push it via stdin rather than splicing strings into the command line.

- ☐ There's no need to worry about hook payloads containing secrets, since the agent's sandbox already redacts them
- ☐ You must commit `.claude/settings.local.json` to share hooks with teammates

Submit

You got all the answers!

 Ask HuggingChat



Question 5: Why can one dashboard support multiple agents?

☒ The same dashboard works for Claude Code, Codex, OpenCode, and Pi because the server normalizes four different payload shapes into one consistent record

Correct! Correct! The ``_normalize`` function maps each platform's field names onto ``{platform, event, tool, args}`` so the UI doesn't need platform-specific branches.

- ☐ The dashboard only works for one platform at a time; you must redeploy it when switching agents
- ☐ OpenCode events are incompatible with the same receiver because OpenCode plugins can't make HTTP calls
- ☐ You have to run a separate dashboard per platform, then merge the logs afterwards

Submit

You got all the answers!

Summary

If you got 4-5 correct, you can wire hooks into a real observability and guardrail flow. If not, review the hands-on example, especially the payload normalization and blocking patterns.

Key Takeaways

- One FastAPI + Gradio process is enough for a lightweight live dashboard
- Hook handlers should fail fast, sanitize payloads, and avoid shell-injection patterns
- A shared receiver works across Claude Code, Codex, OpenCode, and Pi once you normalize the event shape

What's Next

You've finished Unit 5: Hooks. You now have five end-to-end surfaces for shaping how a code agent works: skills, MCP servers, plugins, subagents, and a custom receiver. You can go deeper and build an agent from scratch, so you understand the underlying components.



Ask HuggingChat



plugging into.

`</>` [Update](#) on GitHub

← Hands-On: Agent Activity Dashboard with Gradio

Introduction to Nano Harness →



Ask HuggingChat

